

Núcleo de Pesquisa: Engenharia de Software Assistida por Inteligência Artificial Generativa

Esta é uma pesquisa acadêmica avançada sobre a inserção e as fronteiras da Inteligência Artificial Generativa no escopo completo da Engenharia de Software, fundamentada em estudos publicados entre 2024 e 2026.

A. Delimitação

A Engenharia de Software Assistida por IA Generativa transcende o paradigma restrito de "IA como autocompletar na programação". O escopo desta pesquisa investiga a integração de Modelos de Linguagem de Larga Escala (LLMs) em todo o Ciclo de Vida de Desenvolvimento de Software (SDLC) — englobando levantamento de requisitos, arquitetura, geração e transformação de código, testes automatizados e refatoração de sistemas legados. A delimitação mapeia como essa adoção exige uma formalização metodológica de autonomia (da assistência à execução agentic) para mitigar a introdução de dívida técnica, garantir a manutenibilidade do código-fonte e equilibrar a produtividade com a qualidade arquitetural.

B. Perguntas

1. Como a IA generativa avança entre os papéis de ferramenta auxiliar, geradora de artefatos integrados e agente de programação autônomo?
2. Quais são as evidências empíricas dos efeitos dessa adoção técnica na produtividade, na qualidade e na compreensão do código ao longo do SDLC?
3. Como a IA generativa está sendo incorporada à Engenharia de Software e quais condições organizacionais e processuais precisam ser estabelecidas para que sua utilização seja controlada, verificável e efetiva?

C. Subtemas

- Diferenciação de papéis no desenvolvimento: IA assistiva vs. geração de artefatos vs. IA como agente autônomo (Agentic SE).
- Níveis de Autonomia e Colaboração Humano-IA: supervisão humana interativa vs. execução autônoma (Devin/SWE-agent).

- Impactos ao longo do SDLC: levantamento de requisitos, programação, depuração (debugging), refatoração, testes e documentação.
- Efeitos paradoxais sobre a produtividade, a compreensão arquitetural de longo prazo e a manutenibilidade.
- Dívida técnica provocada por código não revisado e vulnerabilidades geradas por IA.

D. Estratégia e critérios de seleção

Foi realizada uma busca sistemática na literatura internacional de Ciência da Computação (arXiv) e em bases nacionais, incluindo os anais de 2025 do Simpósio Brasileiro de Engenharia de Software (SBES). Priorizou-se o recrutamento de referências de ponta (2024-2026) que apresentassem estudos empíricos, surveys e investigações formais sobre Agentic Development e a evolução contínua da governança de IA. Excluíram-se abordagens genéricas ou de caráter puramente tutorial, focando nas evidências da mudança da carga cognitiva do desenvolvedor (de escritor para revisor/supervisor).

E. Fontes e F. Evidências

Fonte 1

Autor(es): MANDL, P.; MANDL, P.

Ano: 2026

Título: AI-driven Software Development: A Pragmatic Path to Agentic Development Processes

Periódico: arXiv

Tipo de fonte: Framework conceitual / Estudo Metodológico.

Evidências: A fonte mapeia formalmente a progressão da adoção de ferramentas assistivas informais para "Processos de Desenvolvimento Agentic" controlados. O estudo alerta que experimentos controlados revelam tempos maiores de resolução de tarefas para programadores experientes quando tentam depurar hallucinations complexas de IA, evidenciando a necessidade de revisão contínua. (Evidência principal localizada nas Seções 1 e 5 do documento).

Fonte 2

Autor(es): YANG, J. et al.

Ano: 2024

Título: SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering

Periódico: arXiv

Tipo de fonte: Estudo Empírico e Proposta Arquitetural de IA.

Evidências: Demonstra a transição para a IA como agente através da criação de "Interfaces Agente-Computador" (ACI). A fonte prova que modelos não precisam apenas sugerir código estático, mas podem iterar, interagir em ambientes simulados de shell, instalar dependências e testar os próprios artefatos (autonomia de nível Autopilot), alterando a natureza da intervenção humana. (Evidência localizada no Abstract e Introdução).

Fonte 3

Autor(es): CORTI

Ano: 2026

Título: AI assisted Software Engineering: Scaffolding Your Way from One Agent to a Team

Periódico: Corti Engineering Blog / Industry Insight

Tipo de fonte: Estudo da Indústria / Taxonomia.

Evidências: Fornece a classificação pragmática de níveis de adoção e autonomia. Apresenta a parede invisível entre o Copilot (IA imersa nos arquivos, colaboração em tempo real) e o Agent (que requer sandboxing e atua em fluxo passo a passo).

Fonte 4

Autor(es): CRESCENCIO FERREIRA, J. V.; ROLIM DE SOUSA, R.

Ano: 2026

Título: O Impacto da Inteligência Artificial Generativa no Papel do Desenvolvedor de Software

Periódico: RECIMA21 - Revista Científica Multidisciplinar

Tipo de fonte: Revisão Sistemática (Produção Brasileira).

Evidências: Confirma os efeitos da transformação na engenharia de software no cenário nacional. A IAG opera como ferramenta de apoio, expandindo a capacidade técnica do

desenvolvedor, mas, contraditoriamente, inflaciona a exigência por conhecimento de base. Desenvolvedores tornam-se estrategistas focados na validação crítica, e o uso incorreto acarreta perda direta na compreensão da arquitetura.

Fonte 5

Autor(es): BRANCO, B. M. C. et al.

Ano: 2025

Título: Avaliando o Papel do GPT-4o na Aprendizagem de Engenharia de Software: Um Relato de Experiência

Periódico: Anais do Simpósio Brasileiro de Engenharia de Software (SBES)

Tipo de fonte: Relato de Experiência / Estudo Empírico em Educação e SDLC.

Evidências: Aprofunda os resultados sobre o uso de LLMs no Brasil em fases menos óbvias da Engenharia de Software, como o "Levantamento de Requisitos" e "Geração de Testes", exigindo integração humano-máquina para a correção das simulações.

G. Síntese crítica: A Diferenciação de Papéis e Autonomia

A literatura atual categoriza rigorosamente a participação da IA na Engenharia de Software através dos seguintes degraus metodológicos:

1. IA como Ferramenta Auxiliar (Chatbot): O desenvolvimento continua 100% manual e humano. A IA atua num painel lateral, como um oráculo técnico, para sanar dúvidas (ex: "o que causa este erro?") ou ensinar um design pattern. A aplicação no projeto é de inteira responsabilidade do engenheiro de software.
2. IA como Geradora de Artefatos (Copilot): Ferramentas embarcadas no IDE representam o desenvolvimento parcialmente automatizado. A IA antecipa a lógica e gera blocos de código-fonte, suítes de testes unitários ou docstrings (documentação) dinamicamente. Ocorre invariavelmente sob rigorosa supervisão humana, visto que a geração é um autocompletar probabilístico; o modelo não sabe se o código compilará.
3. IA como Agente (Agentic SE): Representa o limiar em direção ao desenvolvimento autônomo. O LLM não apenas gera o texto do artefato, mas atua em uma Interface Agente-Computador (ACI), possuindo um sandbox (VM isolada) onde digita comandos no terminal, navega no repositório, clona dependências, roda testes e executa o debugging de seus próprios erros lógicos (YANG et al., 2024). O engenheiro altera seu

papel de supervisor síncrono para validador assíncrono (ex: revisando o Pull Request final).

H. Composição Acadêmica Única

A Inteligência Artificial Generativa abandonou sua função primária de mero auxílio sintático para atuar na reestruturação integral do Ciclo de Vida do Desenvolvimento de Software (SDLC). O atual estado da arte demonstra que organizações estão escalando do uso individual ad-hoc para metodologias formais de "Desenvolvimento Movido à IA" (AI-driven Software Development) (MANDL; MANDL, 2026). Nesse cenário moderno, a IA transita da abstração e do levantamento de requisitos — processando histórias de usuários — até a geração maciça de rotinas computacionais, a reescrita de testes automatizados e o mapeamento sistêmico de código legado para refatoração.

Entretanto, as promessas empíricas revelam um paradoxo de produtividade: se por um lado o tempo gasto com digitação e boilerplate despenca drasticamente, o tempo necessário para identificação de falhas lógicas (debugging sistêmico) aumenta proporcionalmente (CRESCENCIO FERREIRA; ROLIM DE SOUSA, 2026). A aceitação impensada da geração automática promove uma erosão progressiva do conhecimento de domínio (a equipe perde o contato com o "porquê" a arquitetura foi montada daquela maneira) e insere uma forma velada e exponencial de dívida técnica, frequentemente gerando código com baixo isolamento e potenciais vulnerabilidades de segurança.

Para que a utilização da IA seja metodologicamente controlada, verificável e efetiva, certas condições organizacionais precisam ser estabelecidas. Em primeiro lugar, o papel do engenheiro de software deve ser oficialmente reclassificado de "codificador" para "auditor algorítmico e curador". A colaboração humano-IA deve ser desenhada com fronteiras claras, utilizando contêineres e sandboxes estritos onde a atuação não cause mutações catastróficas em produção. Processos de Desenvolvimento Agentic exigem pipelines de integração contínua (CI/CD) nos quais toda saída da IA (seja como geradora ou como agente) deve, obrigatoriamente, enfrentar ferramentas tradicionais de análise

estática de código e baterias robustas de testes antes de demandar aprovação formal (Code Review) pela contraparte humana.

I. Implicações metodológicas para a TG

Na estruturação da metodologia para o desenvolvimento da plataforma unificada de grade horária acadêmica e alocação de turmas da FATEC, o grupo deverá rejeitar o paradigma de "geração descontrolada" para adotar um fluxo estruturado de desenvolvimento parcialmente automatizado com estrita supervisão humana (nível Copilot e Geração de Artefatos Assistida).

Ao codificar a infraestrutura complexa — particularmente na integração sistêmica do backend em Java 21, mapeamento de objetos usando Hibernate e roteamento via Spring Boot com o banco de dados MySQL — o uso de IA diretamente integrada ao VS Code e o apoio consultivo via Gemini Web para design de arquitetura trarão considerável ganho produtivo. Todavia, como apontado pela literatura, o grupo precisará adotar protocolos rígidos contra a erosão do conhecimento:

1. Micro-Prompting para Refatoração Controlada: Em vez de demandar "Crie um serviço de validação de horário", as interações devem focar em unidades testáveis, solicitando mapeamentos de entidade no banco ou métodos de controle transacional isoladamente.
2. Revisão e Governança: Qualquer script de geração/migração de banco de dados para o MySQL ou regra de negócio específica da matriz da instituição não possuirá o contexto acadêmico prévio na base do LLM. As requisições de código geradas deverão transitar invariavelmente por Pull Requests nominais no GitHub, exigindo o Code Review minucioso de outro integrante da equipe, prevenindo que injeções de SQL ou alucinações nas requisições HTTP do Spring cheguem ao ambiente de validação.
3. Foco na Geração de Testes: Para resguardar a confiabilidade das consultas realizadas pelos alunos da FATEC, a IA generativa será amplamente aproveitada para a criação de suítes de testes automatizados (testes unitários para regras de choque de horários), mitigando a introdução de falhas lógicas na operação educacional.

J. Referências ABNT completas

BRANCO, B. M. C.; MELO NETO, A. L. de; AVELINO, G. A.; MACHADO, V. P. Avaliando o Papel do GPT-4o na Aprendizagem de Engenharia de Software: Um Relato de Experiência. Em: Anais do Simpósio Brasileiro de Engenharia de Software (SBES). Recife: SBC, 2025. p. 470-481. Disponível em: <https://sol.sbc.org.br/index.php/sbes/article/view/37023>. Acesso em: 6 out. 2026.

CORTI. AI assisted Software Engineering: Scaffolding Your Way from One Agent to a Team. Corti Blog, [S. l.], 24 jun. 2026. Disponível em: <https://corti.com/ai-assisted-software-engineering-scaffolding-your-way-from-one-agent-to-a-team/>. Acesso em: 6 out. 2026.

CRESCENCIO FERREIRA, J. V.; ROLIM DE SOUSA, R. O Impacto da Inteligência Artificial Generativa no Papel do Desenvolvedor de Software. RECIMA21 - Revista Científica Multidisciplinar, [S. l.], v. 7, n. 5, p. e757873, maio 2026. DOI: 10.47820/recima21.v7i5.7873.

MANDL, P.; MANDL, P. AI-driven Software Development: A Pragmatic Path to Agentic Development Processes. arXiv, [S. l.], jun. 2026. Identificador: arXiv:2606.15283. Disponível em: <https://arxiv.org/abs/2606.15283>. Acesso em: 6 out. 2026.

YANG, J. et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv, [S. l.], maio 2024. Identificador: arXiv:2405.15793. Disponível em: <https://arxiv.org/abs/2405.15793>. Acesso em: 6 out. 2026.

K. Lacunas

- Apesar dos avanços na literatura de agentes de software (como o SWE-agent), ainda é escassa a produção científica empírica analisando o lock-in (dependência tecnológica) que o nível de abstração dos copilotos gera sobre programadores juniores.
- Há uma lacuna de estudos metodológicos consolidados sobre como enquadrar a supervisão de códigos gerados por IA no escopo estrito de metodologias ágeis em ambientes corporativos e educacionais altamente regulados, assim como métricas efetivas de custo vs. produtividade considerando o desperdício de tempo corrigindo alucinações.